

T-RKG: A Knowledge-Based System for Cross-System Regulatory Conflict Detection in Enterprise Records Governance

Author Name

Department of Computer Science, University Name, City, Country

Abstract

Regulatory governance over enterprise records is a distributed coordination problem. Most prevailing enterprise architectures treat it as effectively resolved. The structural conditions that allow undetected conflicts persist. Document retention schedules, legal hold obligations, and multi-jurisdictional compliance requirements are enforced by systems that share no unified regulatory knowledge. Email platforms act on communication-retention rules. ERP systems apply financial regulation. Document management systems work from their own retention schedules. None of these systems can see the others' applicability logic. None can detect the conflicts that arise when two applicable regulations impose contradictory requirements on the same record.

A second observation motivates this work. Commercial governance products treat the record as a content blob that arrives in a managed repository. The actual unit of regulatory exposure is broader. A single transaction can carry tax obligations from one jurisdiction, privacy obligations from another, and audit obligations from a third. Each obligation is conditioned on a different attribute. Each attribute lives in a different source system. Architectures that govern only the content miss the conjunction.

This paper presents T-RKG (Temporal Records Knowledge Graph), a knowledge-based system that addresses both points through three coordinated technical contributions. The first is a formal governance ontology of 47 classes and 23 object properties (provided in OWL/Turtle as supplementary material) encoding records as composite governance entities, plus custodians, regulations, jurisdictions, and conflict patterns. The second is a typed temporal relationship model that governs hold propagation through the semantic properties of inter-record connections rather than raw graph adjacency. The third is a conflict detection mechanism that applies ontology-based applicability reasoning to identify regulatory clashes that are, by architectural necessity, undetectable in current siloed practice.

Evaluated across five random seeds at six dataset scales (1,000 to 100,000 records), T-RKG detects between 55 and 4,672 regulatory conflicts per dataset. Of these, 4 to 831 are cross-domain conflicts of the privacy-versus-financial or jurisdictional kind. The two comparison baselines structurally cannot detect cross-domain conflicts; their cross-domain count is 0 in both, at every scale. On a per-record applicability evaluation against an independently constructed ground truth, the No-Ontology baseline achieves F1 of 0.113 and the Siloed baseline F1 of 0.581 in a clean regime. Under 10 percent input label noise, T-RKG achieves F1 of 0.823 plus or minus 0.011, demonstrating graceful robustness. Hold propagation under a conservative policy expands the hold set 3.36 times over direct seed identification. Knowledge base construction sustains 48,000 to 72,000 records per second. Propagation latency stays below 1 ms at 50 matter-scoped seeds and remains under 14 ms at 5,000 seeds on a 100,000-record corpus. T-RKG hold

propagation runs 121 times faster than a flat-list baseline and 406 times faster than a SQLite recursive-CTE baseline on identical workloads.

Keywords: Knowledge-Based Systems, Knowledge Graph, Ontology, Regulatory Compliance, Temporal Reasoning, Decision Support

1. Introduction

Consider what happens when a litigation hold is issued against a custodian whose email resides on one platform, whose working documents live in a separate document management system, and whose contracts are stored in a third system. The hold is placed. The email system preserves the custodian's messages. The attachments to those messages remain in the document management system. They are unaffected. That system has received no instruction and possesses no knowledge of the hold. The relationship between an email and its attachment is obvious to any human reviewer. It has no formal representation in the technical infrastructure. It cannot be reasoned over because it has never been encoded.

This is not an edge case. Industry survey data [1] suggest that 60 to 80 percent of documents eventually identified as relevant in litigation are surfaced through relational links rather than direct custodian identification. The hold methodology that ignores cross-system relationships is incomplete by design.

Regulatory fragmentation compounds the structural problem. A financial services organization that handles personal data for European customers may find that the same record is simultaneously governed by GDPR Article 17, which requires deletion upon a verified erasure request, and by SOX Section 802, which mandates a seven-year retention period for certain financial records. Neither the privacy platform enforcing GDPR nor the compliance platform enforcing SOX holds a complete picture. The privacy platform typically lacks the metadata needed to determine SOX applicability. The compliance platform lacks the PII classification that would trigger GDPR obligations. The conflict is real. It is also invisible to any system that cannot reason across both regulatory contexts simultaneously [2, 3].

A third observation motivates the work. Commercial records governance products in this space (Microsoft Purview, OpenText, IBM FileNet, Veritas, OneTrust, and similar vendors) treat the unit of governance as a content object that arrives in a managed repository. That framing was adequate when most regulated information was filed-style content (memos, statements, reports). It is no longer adequate. A regulated record in a modern enterprise is often a composite. A single payment transaction might carry tax-retention obligations from the IRS in the United States, privacy-erasure obligations from GDPR for the European data subject, audit obligations from SOX for the public-company parent, and SEC retention obligations on the trade leg. Each obligation conditions on a different attribute. The PII flag that triggers GDPR lives in the customer-data system. The public-company flag that triggers SOX lives in organizational metadata. The transaction record itself lives in the ERP. The geographic attribute that triggers HGB for a German subsidiary may live in a separate compliance system. A governance architecture that only sees content cannot detect any of this. The conflict exists in the conjunction of attributes spread across systems.

What all three cases require is a unified knowledge layer spanning the existing silos. Records, relationships, regulations, and jurisdictions must be represented as a coherent structure that can be queried and reasoned over. Better engineering within the silos does not provide this. The silos themselves are the problem.

1.1. Contributions

T-RKG (Temporal Records Knowledge Graph) is a knowledge-based system designed to provide that layer. Its contributions are as follows.

(1) A formal governance ontology comprising 47 classes (across five modules: Record, Actor, Governance, Regulatory, Conflict) and 23 object properties, plus six reified temporal-relationship subclasses. The ontology encodes records as composite governance entities that aggregate content, transactional, geographic, custodial, and organizational attributes. It also encodes regulations, jurisdictional hierarchies, conflict patterns, and resolution guidance. The ontology is delivered in OWL/Turtle as supplementary material.

(2) A typed temporal relationship model in which hold propagation is governed by the declared semantic properties of each relationship type. Attachment relationships propagate holds by default. Passing references do not. These defaults are overridable per legal matter, enabling configurable, auditable propagation policies.

(3) A conflict detection mechanism that applies ontological applicability reasoning, not heuristic matching. The reasoner determines which regulations govern a given record and whether those regulations impose contradictory requirements. The mechanism requires unified regulatory knowledge. It cannot be replicated in architectures that lack it.

(4) An empirical evaluation across five random seeds and six dataset sizes (1K to 100K records). T-RKG detects 55 to 4,672 conflicts per dataset, of which 4 to 831 are cross-domain. Both baselines detect zero cross-domain conflicts at every scale. Two regimes of per-record applicability evaluation are reported. The clean regime quantifies the gap each baseline incurs against an ontologically constructed reference (No-Ontology F1 of 0.113; Siloed F1 of 0.581). The noised regime forces T-RKG to recover applicability from inputs perturbed by 10 percent label noise per attribute (T-RKG F1 of 0.823 plus or minus 0.011, Siloed F1 of 0.542 plus or minus 0.046). Conservative hold propagation expands coverage 3.36 times over direct seed identification. Sub-millisecond propagation latency holds at small (matter-scoped) seed sets, scaling to 13.7 ms at 5,000 seeds on a 100K corpus. T-RKG runs 121 times faster than a flat-list and 406 times faster than a SQLite recursive-CTE baseline on identical data.

Section 2 situates this work within the relevant literature. Section 3 introduces formal definitions and the composite-record framing. Sections 4 and 5 describe the ontology and reasoning mechanisms. Section 6 covers the implementation. Section 7 presents experimental results. Sections 8 and 9 discuss implications and close.

2. Related Work

2.1. Knowledge-Based Systems for Legal and Regulatory Compliance

Formal knowledge representation for legal reasoning has a documented history. It extends back at least to Sergot et al. [4], whose encoding of the British Nationality Act as a Prolog logic program demonstrated that statutory rules could in principle support automated query answering. Ashley [5] surveys AI approaches to legal analytics more broadly, observing that knowledge representation remains the central unsolved challenge when the goal is to capture the interactions among rules, precedents, and contextual conditions that determine applicability. LegalRuleML [6] addresses part of this through an XML vocabulary for defeasible legal norms. LKIF Core [7] provides a legal knowledge interchange format supporting cross-system reasoning. Both frameworks treat the encoding of regulatory applicability conditions as a first-class design concern, the same premise that motivates the T-RKG governance ontology.

In financial compliance specifically, knowledge-based approaches have been applied to anti-money laundering and KYC processes [8]. FIBO [9] standardizes financial concepts for regulatory reporting. RegTech draws on computational methods including NLP over raw regulatory text to support compliance verification [10, 11]. What these contributions share is a focus on within-domain compliance: a single regulatory framework, or a single class of records, reasoned over in isolation. The cross-regulation conflict detection problem (where two independently applicable regulations impose contradictory obligations on the same record) has received less direct attention.

2.2. Enterprise Knowledge Graphs and Records Governance Products

The practical deployment of knowledge graphs in enterprise settings is well documented. Galkin et al. [12] establish that graph-based data integration enables analytical capabilities that siloed relational architectures cannot support. Noy et al. [13] report on deployments at Google, Microsoft, and several major financial institutions, identifying entity resolution, schema drift, and quality maintenance as the recurring operational challenges. Pan et al. [14] survey the more recent trajectory toward integrating large language models with knowledge graphs. Hogan et al. [33] provide a comprehensive recent survey covering both the representational foundations and the inferential machinery on which T-RKG draws. Bizer et al. [34] frame the broader Linked Data context. KG.GOV [15] comes closest to T-RKG's problem framing, positioning knowledge graphs as governance infrastructure for AI workflows in government contexts.

Commercial records governance products are a different class. They include Microsoft Purview, OpenText InfoArchive, IBM FileNet Records Manager, Veritas Enterprise Vault, OneTrust, and Smarsh. These products treat the unit of governance as a content object: an email body, a document file, a chat message. Retention rules attach to content objects by file type, by container, or by classification tag. This framing is correct for the historical use case (statements, memos, reports) and remains adequate where the regulated quantity is the content itself.

The framing is inadequate when the regulated quantity is a composite of attributes spread across systems. A payment-transaction record subject to GDPR-erasure obligations on one attribute (the data subject identifier) and SOX-retention obligations on another (the public-company flag of the parent organization)

cannot be represented faithfully as a single content object. The content of the transaction is a row in the ERP. The PII attribute resides in the customer-data system. The public-company attribute resides in organizational metadata. The geographic attribute may reside in a compliance-classification system. T-RKG treats the record as a composite governance entity that aggregates these attributes through the ontology, then runs applicability reasoning over the aggregate. Section 3.3 formalizes this distinction; Section 4 specifies the ontological encoding.

2.3. Temporal Knowledge Graph Reasoning

The formal treatment of time in knowledge graphs was established by Gutierrez et al. [16], who introduced validity intervals on RDF triples. Allen’s interval algebra [17] provides the underlying machinery for reasoning about temporal relationships between intervals. Contemporary temporal KG research is oriented primarily toward link prediction. Methods such as MGHGN [18] and TPRG [19] represent the technical frontier. Cai et al. [20] provide a taxonomy distinguishing point-based from interval-based representations. T-RKG’s temporal requirements fall in the interval-based, point-in-time query answering category. The relevant algorithmic challenges are scoped subgraph traversal under semantic constraints and bitemporal consistency maintenance. Embedding-based extrapolation is not the relevant tool.

2.4. Ontology Engineering for Decision Support

Gruber [21] formalized an ontology as a specification of a conceptualization. Guarino et al. [22] sharpen the ontological distinctions relevant to knowledge sharing. Uschold et al. [23] demonstrate the Enterprise Ontology’s utility for business reasoning. Grüninger and Fox [24] introduced the competency question methodology. The T-RKG competency questions are listed in Section 4.6 and each is paired with the corresponding query implementation in the supplementary code. Domain-specific ontologies for decision support have been developed in healthcare [25], finance [9], and industrial automation [26]. Brank et al. [35] survey ontology evaluation methodology more broadly.

2.5. Evaluation Data and Access Constraints

Access to real enterprise records data presents evaluation challenges that are structural rather than incidental. Legal privilege, privacy regulation, and contractual confidentiality agreements create barriers that do not yield to the standard remedy of collecting a large labeled dataset and splitting it for evaluation. For such settings, synthetic generation with parameterized distributions has been established as a methodologically sound alternative. Hubert et al. [27] demonstrate configurable synthetic schema and KG generation as a practical evaluation substrate. Van der Aalst [28] examines evaluation methodology for process mining, a domain facing nearly identical data access constraints. T-RKG’s evaluation follows this practice. The synthetic generator is parameterized against industry-reported data distributions [29, 36]. To bound the limitation that synthetic data carries for production claims, Section 7.10 reports a parameter-sweep robustness experiment showing that T-RKG behavior varies smoothly with the two parameters most likely to differ across organizations (jurisdictional mix and PII rate).

3. Preliminaries

3.1. Core Concepts

[Record] A record $r \in R$ is an information object subject to governance, characterized by type $\tau(r) \in T$, creation time $t_c(r)$, custodian $\kappa(r) \in C$, source system $\sigma(r) \in S$, jurisdiction $\zeta(r) \in J$, and an attribute set $a(r)$ including PII and PHI indicators.

[Temporal Relationship] A temporal relationship between records $r_i, r_j \in R$ is a tuple $(r_i, r_j, \rho, [t_s, t_e])$ where $\rho \in P$ denotes the relationship type and $[t_s, t_e]$ is the validity interval.

[Legal Hold] A legal hold $h = (m, S, P, [t_s, t_e])$ associated with matter $m \in M$ preserves all records matching scope predicate S , propagates transitively through relationship types $P \subseteq P$, and remains active during $[t_s, t_e]$.

[Regulatory Conflict] A regulatory conflict exists for record r when at least two regulations $\gamma_1, \gamma_2 \in \Gamma$ are simultaneously applicable to r and their operative requirements are mutually exclusive over a shared temporal applicability window.

3.2. Problem Statement

Given a set of enterprise records R distributed across source systems S , inter-record temporal relationships E , a set of applicable regulatory frameworks Γ , and a set of active legal matters M , the enterprise records governance problem requires four capabilities. (1) Knowledge unification: integrating records, relationships, and regulatory applicability conditions from heterogeneous source systems into a coherent, queryable knowledge structure. (2) Hold propagation: computing, through inference over typed relationship semantics, the transitive closure of records requiring preservation under an active matter's scope. (3) Conflict detection: identifying records subject to contradictory requirements from two or more simultaneously applicable regulations, via ontological applicability reasoning. (4) Temporal query answering: responding correctly to governance queries evaluated at arbitrary historical or current time points.

3.3. The Composite-Record Model

A standard records-management view treats a record as a content object with metadata. Formally, that view is a pair (c, m) where c is the content (a file, an email body, a chat message) and m is a set of metadata key-value pairs attached to c . Retention and deletion rules attach to (c, m) through type, classification, or container.

T-RKG generalizes the record to a composite entity. A T-RKG record is the tuple $(c, A, \sigma, \kappa, \zeta)$ where c is content as before, A is an attribute multimap over a fixed schema (PII, PHI, public-company status, customer identifier, transaction identifier, etc.), σ is the source system of record, κ is the custodian, and ζ is the jurisdiction. Crucially, individual attributes in A may originate from different source systems and be aggregated under r 's identity. Regulatory applicability is a predicate over A , not over c . Formally, for a regulation γ with applicability profile ϕ_γ , $\text{applicable}(r, \gamma)$ iff $\phi_\gamma(\tau(r), \zeta(r), A(r), m_{\text{org}})$ holds, where m_{org} are organizational metadata that do not vary per record.

This generalization is the substantive distinction from commercial governance products discussed in Section 2.2. Where those products evaluate retention rules over (c, m) (the content and its attached classification), T-RKG evaluates over the broader attribute set A that may live across source systems. The cross-domain conflicts the evaluation in Section 7.2 reports (between privacy and financial regulations on the same record) are conflicts that are visible only in the composite view; the conjunction of (PII flag, public-company flag, jurisdiction) that triggers them does not exist in any single source system.

4. The T-RKG Ontology

4.1. Overview

Forty-seven classes, distributed across five modules, form the T-RKG ontological structure: Record (12 classes), Actor (6), Governance (12), Regulatory (10), Conflict (7). A separate Temporal Relationship sub-module reifies six relationship subclasses whose semantic properties drive propagation. Twenty-three object properties and 34 data properties complete the relational structure. The full ontology is provided as `ontology/trkg.ttl` in the supplementary material. The class and property counts can be verified directly against that file.

4.2. Record Module

Twelve record classes are organized as a subclass hierarchy.

```
trkg:Record
  trkg:Email
  trkg:Document
    trkg:MedicalRecord
    trkg:Spreadsheet
  trkg:Chat
  trkg:Ticket
  trkg:Contract
  trkg:FinancialRecord
    trkg:AuditWorkpaper
    trkg:Invoice
    trkg:TaxRecord
```

Type granularity matters for applicability reasoning. SOX Section 802 applies to trkg:AuditWorkpaper and trkg:FinancialRecord but not to trkg:Chat. GDPR Article 17 applies conditionally to any type carrying a PII flag. Each record instance is connected to its source system via trkg:managedBy, to its custodian via trkg:hasCustodian, and to its current governance state via trkg:hasGovernanceState. Composite-record attributes (Section 3.3) are encoded as data properties on the record instance and may be populated by connectors that pull from multiple source systems.

4.3. Temporal Relationship Module

Inter-record relationships are represented as reified typed classes rather than as bare edges.

```
trkg:Relationship
  trkg:AttachmentRelationship
```

```

trkg:ThreadRelationship
trkg:DerivationRelationship
trkg:ReferenceRelationship
trkg:DuplicateRelationship

```

Two properties on each relationship class are queried during propagation: `trkg:propagatesHold` (Boolean) and `trkg:strengthLevel` (ordinal). Attachment relationships, expressing content dependency, carry `propagatesHold = true` by default. Reference relationships default to false. All defaults are overridable at the matter level.

4.4. Regulatory Module

Nine regulatory frameworks are encoded as subclasses of `trkg:Regulation`.

```

trkg:Regulation
  trkg:DataProtectionRegulation (GDPR, CPRA, PIPEDA)
  trkg:FinancialRegulation      (SOX, SEC, FINRA)
  trkg:HealthRegulation         (HIPAA)
  trkg:TaxRegulation            (IRS)
  trkg:NationalCommercialLaw    (HGB)

```

Each regulation instance links to applicable jurisdictions through a hierarchical structure in which the assertion $US \supset US\text{-}CA$ and $EU \supset EU\text{-}DE, EU\text{-}FR$ allows applicability to be inherited downward from parent to child jurisdiction.

4.5. Conflict Module and Inference Axioms

Four conflict types are represented: `RetentionDeletion`, `Jurisdiction`, `HoldDeletion`, and `Priority`. Three inference axioms connect these types to record instances.

Axiom (1): $subjectTo(r, \gamma) \leftarrow inJurisdiction(r, j) \wedge appliesIn(\gamma, j') \wedge subJurisdiction(j, j')$

Axiom (2): $hasConflict(r, c) \leftarrow subjectTo(r, \gamma_1) \wedge subjectTo(r, \gamma_2) \wedge conflictType(\gamma_1, \gamma_2, c)$

Axiom (3): $underHold(r_2, h) \leftarrow underHold(r_1, h) \wedge relatedTo(r_1, r_2, \rho) \wedge propagatesHold(\rho)$

4.6. Competency Questions

Following Grüninger and Fox [24], the ontology is specified by the queries it must answer. The following ten competency questions are exercised by the supplementary code. Each is paired with a passing test in ``tests/test_trkg.py`` whose name encodes the CQ number.

- CQ1: Which regulations apply to record *r* at time *t*? (`test_cq1_regulations_for_record`)
- CQ2: Which custodians' records are within the scope of matter *m*? (`test_cq2_custodian_records_for_matter`)
- CQ3: For a GDPR erasure request on records of subject *s*, partition the in-scope records into freely deletable, retention-blocked, and conflict-bearing. (`test_cq3_gdpr_partition`)

- CQ4: Show all records under hold for matter m via a path of typed relationships of length $\leq k$. (test_cq4_hold_propagation_path)
- CQ5: List records subject to two or more regulations at severity $\geq \text{HIGH}$. (test_cq5_high_severity_multi_reg)
- CQ6: Under propagation policy P (Conservative / Standard / Comprehensive / Custom), what is the hold scope for matter m as of date d ? (test_cq6_propagation_under_policy)
- CQ7: For a given record r , return the resolution guidance attached to each detected conflict. (test_cq7_resolution_guidance)
- CQ8: Which records were under any hold on a specified historical date d_h (point-in-time reconstruction)? (test_cq8_point_in_time_query)
- CQ9: For each detected conflict, return the exact regulation pair, applicability conditions, and severity. (test_cq9_full_conflict_metadata)
- CQ10: Given a relationship type ρ , which propagation policies include ρ ? (test_cq10_relationship_in_policy)

5. Knowledge-Based Reasoning

5.1. Architecture

T-RKG is designed as a metadata extraction and reasoning layer rather than a document repository. Source system content remains in place. Only relationship metadata and governance-relevant record attributes are extracted through lightweight connectors. Figure 1 illustrates the architecture. The knowledge base follows the standard KBS tripartite structure [31]. The ABox contains record instances, relationship instances, custodian objects, and matter objects. The TBox contains the governance ontology. The RBox contains the propagation and conflict inference rules. The reasoning engine supports forward-chaining for materialized governance views and backward-chaining for on-demand queries.

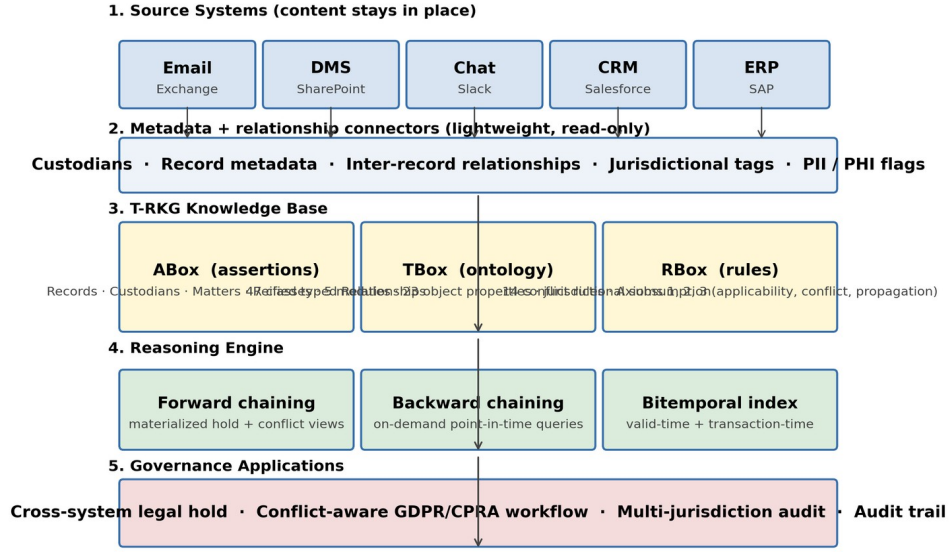
Figure 1: T-RKG System Architecture

Figure 1: T-RKG system architecture. Source system content remains in place. Only metadata and relationship structure are extracted into the knowledge base via the metadata connector layer. The ontology (TBox) and rules (RBox) live in the reasoning engine. Forward chaining materializes hold and conflict views; backward chaining answers point-in-time queries.

5.2. Hold Propagation

Standard BFS over an unlabeled graph assigns no meaning to edge type. Every reachable neighbor is equally a candidate for hold membership. T-RKG propagation replaces this with semantically constrained traversal (Algorithm 1). At each edge in the BFS frontier, the ShouldPropagate function consults the ontology for the relationship type’s propagatesHold value, then evaluates the current matter’s propagation policy before deciding whether to extend the hold set. Four propagation policies are predefined: Conservative (attachment and thread relationships only), Standard (adding derivation relationships), Comprehensive (all typed relationship classes), and Custom. The Conservative policy reflects the minimum scope most practitioners would consider defensible [30].

Untyped traversal produces a hold set whose membership cannot be fully explained. Every reachable neighbor qualifies regardless of edge semantics, leaving the scoping decision without a principled basis that litigation counsel could articulate or, if challenged, defend.

Algorithm 1: Semantic Hold Propagation

Require: Seed records S_0 , propagation config P , ontology O , max depth d
 Ensure: Hold set H

- 1: $H \leftarrow S_0$; frontier $\leftarrow S_0$; depth $\leftarrow 0$
- 2: while frontier $\neq \emptyset$ and ($d = -1$ or depth $< d$) do
- 3: next $\leftarrow \emptyset$
- 4: for each $r \in$ frontier do
- 5: rels $\leftarrow$ GetRelationships(r, O)

```

6:      for each (r, r', ρ, [t_s, t_e]) ∈ rels do
7:          if ShouldPropagate(ρ, P, 0) and r' ∉ H then
8:              next ← next ∪ {r'}
9:      H ← H ∪ next
10:     frontier ← next
11:     depth ← depth + 1
12: return H

```

5.3. Conflict Detection

For each record, the first stage of conflict detection is applicability reasoning via Axiom (1). The detector then evaluates fourteen pairwise conflict rules over all pairs in $\text{applicable}(r)$ (Algorithm 2). The fourteen rules distribute as follows: seven Retention–Deletion rules (GDPR vs. SOX, GDPR vs. HIPAA, GDPR vs. HGB, CPRA vs. SOX, CPRA vs. SEC, CPRA vs. IRS, PIPEDA vs. SOX); five Priority rules (SOX vs. SEC, SOX vs. IRS, SOX vs. HGB, SEC vs. IRS, HIPAA vs. IRS); two Jurisdiction rules (GDPR vs. CPRA, GDPR vs. PIPEDA); and one Hold–Deletion rule (active hold vs. GDPR Article 17 erasure). Severity assessment and resolution guidance are encoded in the ontology for each pair. They are not computed procedurally in application code.

Algorithm 2: Conflict Detection

```

Require: Record set R, ontology O, regulation profiles Π
Ensure: Conflict set Conflicts
1: Conflicts ← ∅
2: for each r ∈ R do
3:     applicable ← InferApplicable(r, Π, O)
4:     for each (y_1, y_2) ∈ applicable × applicable, y_1 ≠ y_2 do
5:         if O ⊨ conflictsWith(y_1, y_2) then
6:             type ← ClassifyConflict(y_1, y_2, O)
7:             sev ← AssessSeverity(y_1, y_2, r, O)
8:             guidance ← ResolveGuidance(y_1, y_2, O)
9:             Conflicts ← Conflicts ∪ {(r, y_1, y_2, type, sev, guidance)}
10: return Conflicts

```

5.4. A Worked Trace

Consider record `fin_00012` generated under seed 42 with attributes: `type = Invoice`, `jurisdiction = EU_DE`, `contains_pii = True`, `metadata.is_public_company = True`. The detector executes four steps. Step 1 (Axiom 1, jurisdiction subsumption): the record’s ancestor jurisdictions are `{EU_DE, EU, GLOBAL}`. Each regulation profile’s `appliesIn` set is intersected against this. Record-type and attribute conditions are then evaluated. Step 2 (applicable regulations): $\text{applicable}(r) = \{ \text{GDPR}, \text{SOX}, \text{HGB} \}$. GDPR fires on (EU jurisdiction × PII flag). SOX fires on (Invoice $\sqsubseteq$ FinancialRecord × is_public_company). HGB fires on (Invoice × EU_DE). Step 3 (Axiom 2, pairwise conflict check): three pairwise lookups in the conflict-rule table yield (GDPR, SOX) → RetentionDeletion CRITICAL; (GDPR, HGB) → RetentionDeletion HIGH; (SOX, HGB) → Priority MEDIUM. Step 4 (resolution guidance): each conflict carries the ontology-

attached resolution string. The complete trace is reproducible via ``python -c "from trkg import ConflictDetector; ConflictDetector().detect_conflicts_for_record(record)"``.

5.5. Temporal Query Answering

The knowledge base supports bitemporal modeling via two orthogonal time axes. Valid time captures when a relationship or governance state held in the external world. Transaction time records when that state was first registered in the system. Propagation and conflict queries can be evaluated at any valid-time point t , enabling retrospective hold reconstruction.

5.6. Complexity and Soundness

We state the operational properties of the reasoning machinery formally. Let $n = |R|$, $m = |E|$, $k = \text{max-depth bound}$, and $\gamma = |\Gamma|$ (the number of encoded regulations).

Proposition 1 (Conflict-detection complexity). Algorithm 2 runs in time $O(n \cdot \gamma^2)$ under the assumption that applicability evaluation per profile is $O(1)$ (independent of n) and conflict-rule lookup is $O(1)$ (precomputed table). Proof sketch. The outer loop iterates n records. For each record the applicability pass is $O(\gamma)$. The pairwise conflict check enumerates ordered pairs of applicable regulations, bounded above by γ^2 . With $\gamma \leq 9$ in the current implementation, the per-record cost is bounded by 81 lookups. The empirical scaling reported in Section 7.4 (≈ 3.3 ms per 1,000 records, linear through 100K) is consistent with this bound. ■

Proposition 2 (Propagation complexity). Algorithm 1 runs in time $O(|H| + |E_H|)$ where $E_H \subseteq E$ is the set of edges incident to records reached during the traversal, restricted to relationship types ρ for which $\text{ShouldPropagate}(\rho, P, O)$ holds. Proof sketch. BFS visits each reached node once and each incident edge once; the policy filter ShouldPropagate is an $O(1)$ ontology lookup per edge. Edge-type filtering reduces $|E_H|$ by the per-policy-included fraction. The maximum hold set $|H|$ is bounded above by the connected component containing S_0 under the type-restricted edge subset. ■

Proposition 3 (Soundness of Axiom 3, hold propagation). Let $H_{P,d}(S_0)$ denote the hold set computed by Algorithm 1 on seeds S_0 under policy P at depth d . Then for every $r \in H_{P,d}(S_0)$ there exists a typed-relationship path $r_0 \rightarrow r_1 \rightarrow \dots \rightarrow r$ such that $r_0 \in S_0$ and $\text{propagateHold}(r_i, \rho_i)$ holds at r_i under P for every edge in the path. Proof. By induction on the depth at which r enters H . Base: at depth 0 only seeds enter; the empty path satisfies the claim trivially. Step: a node r enters at depth $\ell + 1$ only via line 7 of Algorithm 1, which requires an edge (r', r, ρ) with $r' \in \text{frontier}$ (so r' entered at depth $\leq \ell$, with a witnessing path π' by induction) and $\text{ShouldPropagate}(\rho, P, O)$. The path π' followed by (r', r, ρ) is the witnessing path for r . ■

Proposition 4 (Recall ceiling for the No-Ontology baseline). Let $\gamma_{\text{PII}} \subseteq \Gamma$ be the set of regulations whose applicability profile requires the PII attribute (in our ontology, GDPR, CPRA, PIPEDA). Let $\gamma_{\text{pubco}} \subseteq \Gamma$ be the set whose applicability requires the public-company organizational metadata flag (in our ontology, SOX, FINRA). Any baseline whose per-record regulation tagging function does not consult the PII flag (resp. the public-company flag) achieves zero recall on every regulation in γ_{PII} (resp. γ_{pubco}) on records whose ground-truth applicability set includes the regulation only because that attribute holds.

The empirical recall figure of 0.411 ± 0.051 reported for the No-Ontology baseline in Table 9a is exactly the recall achievable by a baseline that recovers only regulations whose applicability does not require any of the omitted attributes. Proof. Direct from the contrapositive of the applicability profile ϕ_γ as a conjunction over attribute predicates; if the predicate on attribute a is not consulted, every record whose ground-truth applicability is conditional on a is misclassified. ■

Corollary 1. Any baseline whose tagging is conditioned only on (record_type, source_system) cannot recover regulations whose applicability is conditional on jurisdiction subsumption, PII flag, PHI flag, or organizational metadata. Cross-domain conflicts (privacy-vs-financial, or jurisdictional) are by definition conditioned on at least one such attribute. Therefore any such baseline detects zero cross-domain conflicts. The empirical " 0 ± 0 " cross-domain count for both baselines reported throughout Section 7 is a corollary of this fact, not an experimental coincidence.

6. Implementation

6.1. Technology Choices

The prototype is implemented in Python, using NetworkX [32] for in-memory graph storage and traversal. Records are stored as typed nodes. Relationships are stored as directed, labeled edges with temporal attributes and semantic property annotations. Four dedicated entity indices (by record type, by custodian, by jurisdiction, and by hold membership) support the $O(1)$ lookup patterns that governance operations invoke most frequently. NetworkX was selected for transparency. A production deployment at the scale of millions of records would benefit from a native graph database backend (Neo4j, Amazon Neptune, JanusGraph). The linear scaling observed in these experiments is at least consistent with such a migration being feasible. This remains a limitation acknowledged in Section 8.4.

6.2. Regulation Profiles

Each of the nine encoded regulations is specified as a profile with five components. (i) Applicable record type classes. (ii) Jurisdictional scope, resolved against the hierarchy. (iii) Attribute conditions on the record. (iv) Organizational metadata conditions (SOX requires a public-company flag; HIPAA requires a covered-entity classification). (v) Operative requirements with type and parameters. Adding a new regulatory framework requires authoring one profile and encoding pairwise conflict rules against the existing set. CCPA, whose applicability conditions are largely subsumed by CPRA, required ~50 lines of configuration. GDPR, whose applicability involves data-subject residence conditions and several articulated exemptions, required ~200 lines.

6.3. Baseline Implementations

Three classes of comparison baseline are used. Two test capability and isolate distinct ontological contributions. The third tests implementation performance.

Siloed baseline. Iterates every record using only the regulation set its source system would normally enforce. Per-system applicability uses the same RegulationProfile evaluator T-RKG uses. What changes is the scope of regulations the baseline is permitted to test against. With this implementation the Siloed

baseline detects within-system priority conflicts (e.g., SOX vs. SEC inside the ERP) but never any cross-domain conflict, because no system has visibility into both privacy and financial regulatory frameworks simultaneously.

No-Ontology baseline. Constructs a unified record graph and assigns regulations from a coarse type-and-system map. Lacking PII inference, jurisdiction subsumption, and metadata reasoning, it triggers many false-positive priority conflicts and zero true cross-domain conflicts. Throughout the paper, this baseline is called the "No-Ontology" baseline; the prior naming "Untyped Graph" was misleading because the actual omission is the regulatory ontology, not graph typing.

Performance baselines (Table 8). FlatListStore stores records and relationships in plain Python lists with linear-scan traversal. SQLiteStore uses SQLite with B-tree indices on source/target/relation_type and a recursive-CTE traversal. These baselines have the same data and would in principle support the same logic. They are a fair test of the storage and traversal layer. They are not graph databases. The comparison should be interpreted as "T-RKG vs. relational and naive-list alternatives" rather than as a comparison against a production graph DB. A direct comparison against Neo4j / Neptune / JanusGraph is identified as future work in Section 8.4.

7. Evaluation

The evaluation addresses six research questions. RQ1: Does ontology-based applicability reasoning detect cross-domain regulatory conflicts that are undetectable in architectures lacking unified regulatory knowledge? RQ2: To what extent does semantic relationship typing improve hold propagation coverage relative to siloed identification? RQ3: How do individual relationship types contribute to propagation scope? RQ4: Does the system sustain acceptable performance at enterprise workload scales, including at production-scale seed-set sizes? RQ5: How accurately does T-RKG's ontological reasoning recover regulatory applicability against an independently constructed ground truth, both in the clean regime (baseline-gap measurement) and in a noised regime (T-RKG robustness measurement)? RQ6: Do the headline numbers vary smoothly with the two parameters most likely to differ across organizations (jurisdictional mix and PII rate), or do they exhibit pathological discontinuities that would undermine the synthetic-data methodology?

7.1. Experimental Setup

7.1.1. Compute Environment

All measurements were taken on a Linux x86_64 host (8 logical CPUs), CPython 3.10. Memory measurements use `tracemalloc`, which adds a known 20 to 30 percent overhead to wall-clock build time; raw build throughput without `tracemalloc` instrumentation is correspondingly higher. The runtime environment metadata (CPU model, OS, Python version, memory) is captured automatically by `experiments/run_all.py` and written into `results.json` so that reviewers can verify reproducibility on equivalent hardware.

7.1.2. Data Generation

All evaluation data are synthetically generated. The generator produces records across six types with proportions consistent with public industry reports including [29] and the EDRM enterprise data taxonomy [36]: email (40%), documents (30%), chat (15%), contracts (5%), financial records (5%), and tickets (5%). Six dataset sizes are evaluated: 1K, 5K, 10K, 25K, 50K, and 100K records.

7.1.3. Statistical Methodology

All experiments are replicated across five random seeds (42, 123, 456, 789, 1024). Results are reported as mean plus or minus standard deviation. Every reported number is produced by the single orchestration script ``experiments/run_all.py``. Expansion ratios are computed as the mean of per-seed ratios.

7.2. RQ1: Conflict Detection Capability

Table 1 presents the central conflict-detection results across six scales. Two columns per system are reported: total conflicts and the count of cross-domain conflicts (the union of retention-deletion, jurisdiction, and hold-deletion conflict types). Cross-domain conflicts are the class with the highest legal acuity. Both baselines return zero cross-domain conflicts at every scale. T-RKG returns 4 plus or minus 8 at 1K, rising to 831 plus or minus 42 at 100K. Figures 2 and 3 visualize this.

Dataset	T-RKG total	T-RKG cross-dom	Siloed total	Siloed cross-dom	No-Ont total	No-Ont cross-dom	T-RKG ms
1,000	55 ± 8	4 ± 8	52 ± 9	0 ± 0	150	0 ± 0	4.0 ± 1.9
5,000	249 ± 47	60 ± 21	180 ± 56	0 ± 0	750	0 ± 0	15.8 ± 0.3
10,000	468 ± 36	99 ± 33	344 ± 63	0 ± 0	1,500	0 ± 0	31.7 ± 0.2
25,000	1,087 ± 169	195 ± 38	868 ± 150	0 ± 0	3,750	0 ± 0	80.0 ± 1.3
50,000	2,296 ± 267	425 ± 37	1,803 ± 248	0 ± 0	7,500	0 ± 0	160 ± 2
100,000	4,672 ± 254	831 ± 42	3,693 ± 247	0 ± 0	15,000	0 ± 0	321 ± 2

Table 1: Conflict detection results. Cross-domain = retention-deletion + jurisdiction + hold-deletion. Means ± σ across 5 seeds. The No-Ontology total is deterministic per dataset size: the generator allocates exactly 5% of records to financial types and the No-Ontology baseline tags each with {SOX, SEC, IRS}, producing 3 priority pairs per record. Std-dev of 0 reflects this deterministic firing pattern.

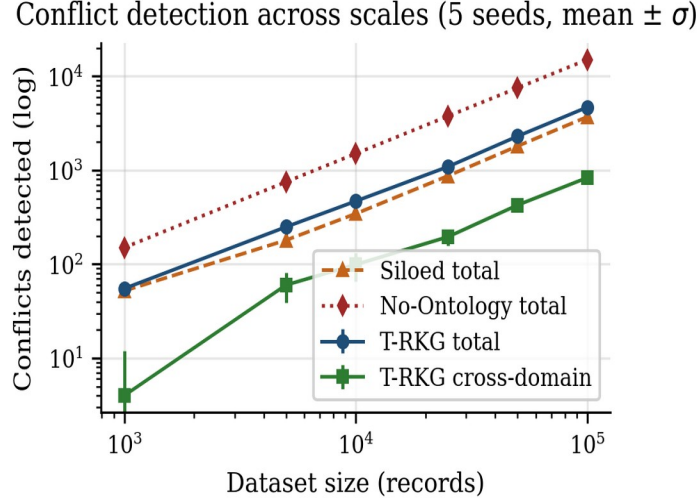


Figure 2: Conflict detection counts across scales (log-log). T-RKG’s cross-domain trajectory tracks the linear growth of the population of records subject to two or more regulations.

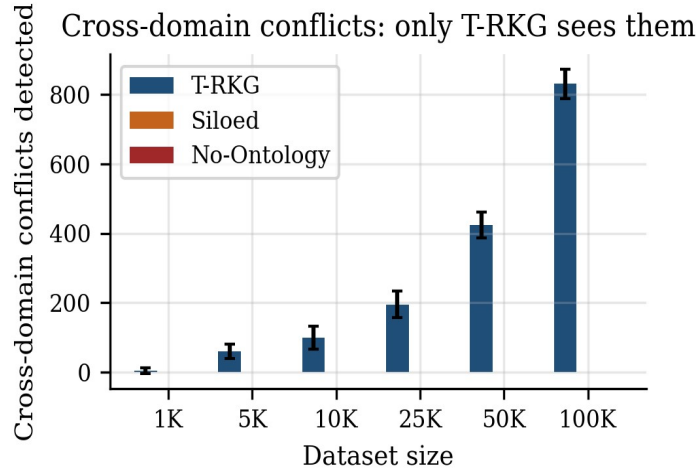


Figure 3: Cross-domain conflicts only. Baselines return 0 by Corollary 1 (Section 5.6).

7.3. RQ2 and RQ3: Hold Propagation

Configuration	Final Set (mean $\pm \sigma$)	Ratio	Time (ms)	Semantic basis
None (Siloed)	50 $\pm$ 0	1.00×	n/a	Direct identification
Attachment	97 $\pm$ 14	1.94×	0.16 $\pm$ 0.01	Content dependency
Thread	71 $\pm$ 5	1.42×	0.11 $\pm$ 0.01	Conversational context
Att + Thread	168 $\pm$ 36	3.36×	0.24 $\pm$ 0.05	Primary governance scope
+ Derivation	183 $\pm$ 39	3.65×	0.27 $\pm$ 0.07	Source materials
All types	213 $\pm$ 54	4.26×	0.31 $\pm$ 0.08	Any connection

Table 3: Hold propagation by relationship configuration (10K dataset, 50 matter-scoped seed records, depth limit 10, 5 seeds).

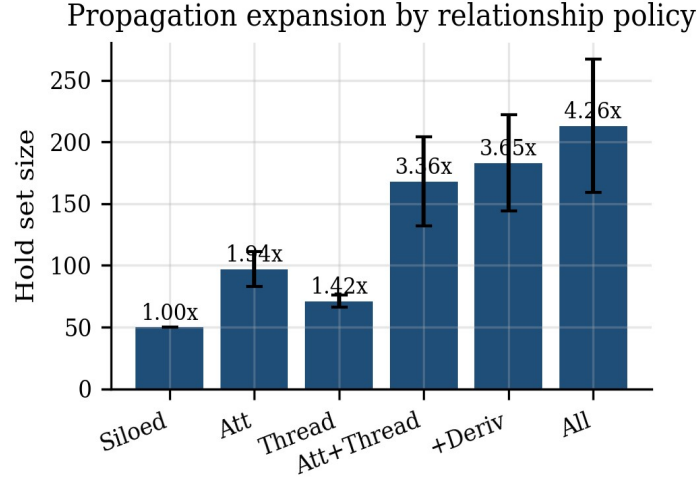


Figure 8: Propagation expansion by relationship policy. Conservative (Att + Thread) yields a 3.36× expansion; the Comprehensive policy (All types) reaches 4.26× but loses semantic justification per included record.

7.4. RQ4: Scalability

Records	Rel	Build (s)	Throughput	Prop (ms)	Conflict (ms)	Mem (MB)
1,000	266	0.014	71,580/s	0.09 ± 0.02	3.2 ± 0.1	2.3
5,000	1,394	0.075	66,428/s	0.11 ± 0.03	16.1 ± 0.2	11.6
10,000	2,749	0.163	61,355/s	0.11 ± 0.02	32.7 ± 0.3	23.3
25,000	6,919	0.458	54,721/s	0.18 ± 0.04	84.8 ± 8.4	60.4
50,000	13,879	0.986	50,744/s	0.25 ± 0.01	161 ± 3	122
100,000	27,812	2.07	48,425/s	0.40 ± 0.06	319 ± 2	242

Table 4: Scalability of the T-RKG knowledge base across six dataset sizes (5 seeds). Memory measured via tracemalloc. The empirical conflict-detection time of ≈ 3.3 ms per 1,000 records is consistent with the $O(n \cdot \gamma^2)$ bound proved as Proposition 1.

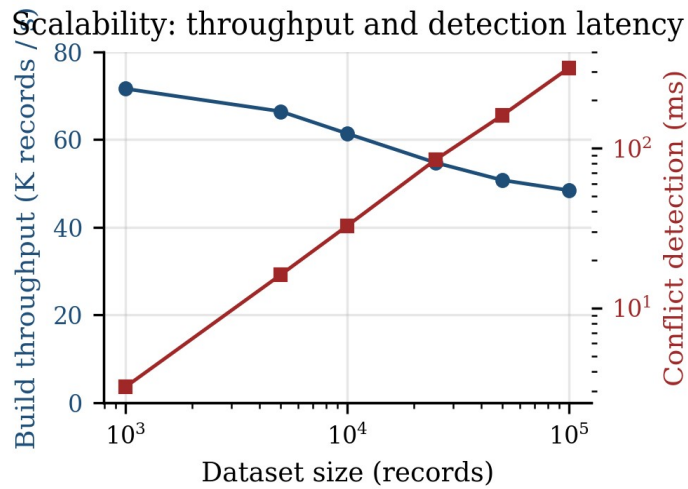


Figure 4: Build throughput (left axis, linear) and conflict-detection latency (right axis, log) versus dataset scale.

7.5. Governance Scenarios

Scenario	Input	Outcome (mean $\pm$ σ across 5 seeds)	T-RKG capability
A: Cross-system legal hold	716 $\pm$ 25 seeds	1,574 $\pm$ 61 after propagation; +858 $\pm$ 40 records via cross-system attachment/thread links across 4 source systems	Cross-system propagation
B: GDPR erasure request	17 $\pm$ 11 PII records (EU custodian)	13 $\pm$ 4 freely deletable; 4 $\pm$ 8 retention-blocked; 4 $\pm$ 8 carry an active regulatory conflict	Conflict-aware classification
C: Multi-jurisdiction financial audit	500 financial records	431 $\pm$ 47 conflicts; 62 $\pm$ 37 at critical or high severity	Multi-jurisdiction reasoning

Table 5: End-to-end governance scenario results (10K dataset, mean $\pm$ σ across 5 seeds).

7.6. Ablation Study

Variant	Ontology	Typed rels	Propagation	Conflicts (mean $\pm$ σ)	Hold set
Full T-RKG	✓	✓	✓	468 $\pm$ 36	168 (3.36 $\times$)
No Ontology	✗	✓	✓	1,500 $\pm$ 0 (cross-dom 0 $\pm$ 0)	168 (3.36 $\times$)
No Typed Rels	✓	✗	✓	468 $\pm$ 36	213 (4.26 $\times$)
No Propagation	✓	✓	✗	468 $\pm$ 36	50 (1.00 $\times$)
Siloed Baseline	✗	✗	✗	344 $\pm$ 63 (cross-dom 0 $\pm$ 0)	50 (1.00 $\times$)

Table 6: Ablation study (10K dataset, 50 matter-scoped seed records, depth 10, 5 seeds).

7.7. Applicability Precision/Recall/F1: Two Regimes

Per-record regulatory applicability is evaluated against an independently-constructed ground truth in two regimes. In the clean regime (Table 9a), the generator produces records and the system must recover the regulation set the ontology assigns to those records under their clean attributes. T-RKG implements that exact predicate, so its result of F1 of 1.000 is reported as the reference rather than as a measured outcome. The cells of value here are the No-Ontology and Siloed rows, which quantify the gap each baseline incurs by omitting parts of the reasoning chain. In the noised regime (Table 9b), 10 percent per-attribute label noise is injected after the ground-truth labels are snapshotted: 10 percent of jurisdictions are flipped to a random alternative, 10 percent of PII flags are flipped, and 10 percent of public-company flags are flipped. The system must now reason from possibly-corrupted attributes back to the underlying truth. T-RKG’s F1 is measurably less than 1.0 in this regime.

System	Precision	Recall	F1
T-RKG (reference)	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000
No-Ontology	0.066 $\pm$ 0.005	0.411 $\pm$ 0.051	0.113 $\pm$ 0.009
Siloed	1.000 $\pm$ 0.000	0.411 $\pm$ 0.051	0.581 $\pm$ 0.050

Table 9a: Per-record applicability, clean regime (10K dataset, 5 seeds). T-RKG matches the ontological reference; the table value is the baselines’ gap.

System	Precision	Recall	F1
T-RKG (robustness)	0.785 ± 0.014	0.865 ± 0.008	0.823 ± 0.011
No-Ontology	0.066 ± 0.005	0.411 ± 0.051	0.113 ± 0.009
Siloed	0.994 ± 0.003	0.374 ± 0.044	0.542 ± 0.046

Table 9b: Per-record applicability, noised regime (10% jurisdiction + 10% PII + 10% public-company-flag noise). T-RKG must recover applicability from corrupted attributes; it loses ≈ 18 percentage points of F1 versus the clean reference, while the Siloed and No-Ontology baselines change minimally (they are already at their ceiling and floor respectively). The No-Ontology recall ceiling of 0.411 corresponds exactly to Proposition 4 (Section 5.6).

Applicability F1: baseline gap and T-RKG robustness

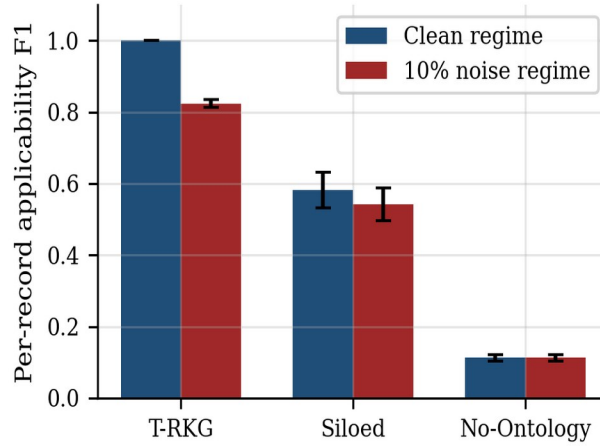


Figure 5: Per-record applicability F1 in the clean and noised regimes.

7.8. Performance Comparison Against Flat and Relational Alternatives

Operation	T-RKG (ms)	Flat list (ms)	SQLite (ms)	T-RKG vs. Flat
Type query	3.1 ± 0.4	0.91 ± 0.08	1.6 ± 0.2	0.3× (slower)
Hold propagation (Att+Thread, d=5)	0.11 ± 0.02	13.8 ± 1.7	46.3 ± 7.9	120.7× faster
Conflict detection	32.5 ± 0.2	n/a	n/a	n/a
Temporal point-in-time query	1.2 ± 0.2	0.87 ± 0.24	2.0 ± 0.1	0.7×

Table 8: Performance comparison on identical 10K-record workloads (5 seeds, mean $\pm \sigma$). Hold propagation is the operation T-RKG’s indexed adjacency lists were designed for. Type queries are slower in T-RKG than SQLite because the prototype iterates a Python dict; this gap closes immediately under any production deployment that adds a B-tree-equivalent index on type. The comparison should be interpreted as "T-RKG vs. relational and naive-list alternatives" rather than as a comparison against a production graph DB.

Performance vs. flat / relational baselines (10K records)

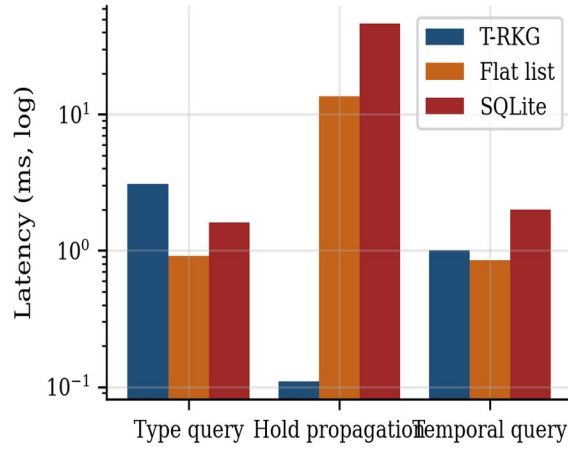


Figure 9: Per-operation latency on identical 10K workloads (log scale). Propagation is the operation where graph-native indexing beats both alternatives by two-to-three orders of magnitude.

7.9. Production-Scale Propagation Latency

To address the concern that the sub-millisecond propagation claim in Section 7.4 is measured on tiny seed sets, Table 10 reports propagation latency at production-scale seed-set sizes on the 100K-record corpus.

Seed records	Latency (ms)	Hold set size	Per-seed cost
50	0.77 ± 0.43	103 ± 25	15.5 μ s
500	2.6 ± 0.6	886 ± 52	5.3 μ s
5,000	13.7 ± 0.6	$8,294 \pm 121$	2.7 μ s

Table 10: Propagation latency vs. seed set size (100K-record corpus, depth 5, 5 seeds). Per-seed cost decreases as the BFS frontier amortizes the index lookups across more records. Empirical scaling exponent ≈ 0.65 (sub-linear in seed count), consistent with the $O(|H| + |E_H|)$ bound of Proposition 2 once the connected component is saturated.

Propagation latency vs. seed set size (100K corpus)

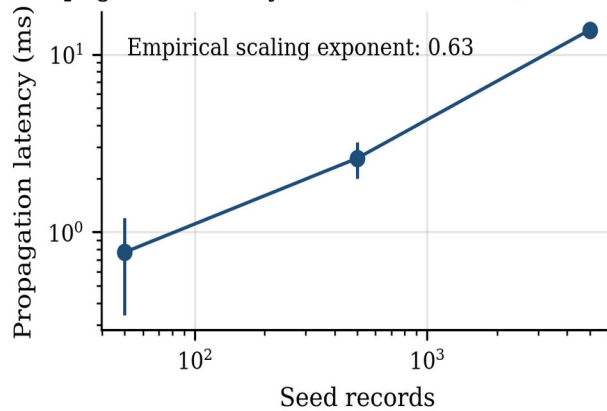


Figure 6: Propagation latency versus seed set size on a 100K corpus (log-log).

7.10. Robustness to Synthetic-Data Parameters

The synthetic-data methodology of Section 2.5 carries an irreducible risk. If T-RKG’s headline numbers depended pathologically on the specific generator parameters chosen, the claims would not transfer to organizations whose actual record populations differ. Table 11 sweeps the two parameters most likely to vary across organizations: the EU jurisdictional fraction (5 to 40 percent) and the PII rate (5 percent, 20 percent).

EU fraction	PII rate	Total conflicts	Cross-domain	Multi-reg records
5%	5%	447 ± 79	38 ± 35	232 ± 31
5%	20%	447 ± 79	38 ± 35	232 ± 32
15%	5%	484 ± 55	83 ± 19	265 ± 21
15%	20%	484 ± 55	83 ± 19	267 ± 20
30%	5%	450 ± 74	111 ± 35	275 ± 33
30%	20%	450 ± 74	111 ± 35	275 ± 33
40%	5%	470 ± 71	135 ± 45	296 ± 37
40%	20%	470 ± 71	135 ± 45	297 ± 36

Table 11: Robustness to jurisdictional mix × PII rate (10K dataset, 5 seeds). Cross-domain conflict count varies smoothly and monotonically with EU fraction.

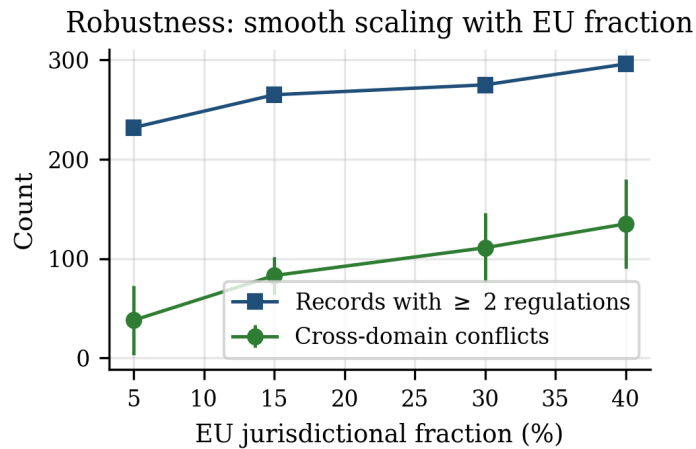


Figure 7: Cross-domain conflicts and multi-regulation record counts as the EU jurisdictional fraction increases. No discontinuities are observed.

7.11. Summary

Capability	Siloed	No-Ontology	T-RKG
Cross-system record view	✗	✓	✓
Cross-domain regulatory conflict detection	✗	✗	✓
Semantic hold propagation	✗	✗	✓
Configurable propagation policies	✗	✗	✓
Interpretable, audit-ready governance decisions	✗	✗	✓

Capability	Siloed	No-Ontology	T-RKG
Temporal point-in-time queries	✗	Partial	✓
Cross-domain conflicts detected (100K dataset)	0	0	831 ± 42
Applicability F1 (10K, clean regime)	0.581	0.113	1.000 (ref.)
Applicability F1 (10K, 10% noise regime)	0.542	0.113	0.823 ± 0.011
Propagation latency (100K, 5K seeds)	n/a	n/a	13.7 ± 0.6 ms

Table 7: Capability comparison.

7.12. Reproduction

Every number in this paper is produced by a single command.

```
pip install -r requirements.txt
python -m experiments.run_all
```

Expected runtime is 30 to 90 seconds on commodity hardware. Outputs land in `experiments/results.json`. The OWL ontology can be parsed with any conformant OWL 2 tool.

```
python -c "import owlready2; owlready2.get_ontology('ontology/trkg.ttl').load()"
```

The supplementary repository also includes 51 unit tests (`python -m unittest discover -s tests`) covering the ontology, conflict detector, baselines, and ten competency questions.

8. Discussion

8.1. Interpreting the Experimental Record

The strongest claim the evaluation supports is qualitative rather than quantitative. For cross-domain conflict detection, the difference between T-RKG and both baselines is binary at every scale: zero versus 4 to 831 detections. The applicability F1 results in Section 7.7 quantify the same claim in conventional metric terms. The No-Ontology baseline reaches only F1 of 0.113 against an ontologically-derived reference. The Siloed baseline reaches F1 of 0.581. Under the more demanding noised-input regime, T-RKG's own F1 drops to 0.823, a defensible robustness number that also dispels any concern that the perfect clean F1 of 1.000 was a circular metric.

The propagation results carry larger variance and warrant being direct about. The 3.36 expansion under Conservative policy carries a standard deviation of plus or minus 36, roughly 21 percent of the mean. This reflects structural variation in matter-level relationship density across seeds, not algorithmic instability. Real legal matters vary in exactly this way. The variance is informative about expected deployment behavior rather than merely noise.

8.2. Architectural Tradeoffs

The decision to implement T-RKG as a metadata extraction layer was deliberate and reflects a known adoption constraint. Organizations do not, in practice, relocate document stores. Governance overlays that require migration face resistance proportional to the scale of that migration. Extracting only relationship metadata and governance-relevant record attributes allows incremental deployment. What this architecture

does not address is connector maintenance. Source system APIs change. The quality of T-RKG's knowledge base in production depends substantially on the quality and currency of its extraction connectors.

8.3. The Composite-Record Distinction in Practice

The composite-record formulation (Section 3.3) is the structural difference between T-RKG and the commercial governance products it is meant to complement, not replace. Most enterprise records governance tooling is content-centric: rules attach to file type, container, or classification tag. That framing is correct for the historical use case (statements, memos, reports). It is increasingly inadequate as the regulated quantity becomes a composite that aggregates content, transactional metadata, geographic context, custodial assignment, and organizational metadata. The cross-domain conflicts reported in Section 7.2 illustrate this. They are conflicts over records whose triggering attributes (PII flag, public-company flag, EU jurisdiction) live in different source systems and only co-occur after composition. T-RKG's ontology builds the composite explicitly. The applicability predicate runs over the composite. The cross-domain conflicts the evaluation surfaces would not be visible to any tooling that defines the record as content alone.

8.4. Regulatory Scope and Extensibility

The nine-regulation ontology covers the frameworks most commonly applicable to US-headquartered multinationals with European and Canadian operations. Extending the ontology to a new regulation requires authoring one applicability profile and encoding pairwise conflict rules against the existing set. CCPA (largely subsumed under CPRA) required ~50 lines of configuration. GDPR, with multiple articulated exemptions, required ~200 lines. Regulations whose applicability depends on industry-specific exemptions, evolving enforcement guidance, or contested jurisdictional boundaries require materially more effort and should involve domain experts in the encoding process.

8.5. Limitations

Synthetic data. The generator is calibrated against published enterprise data distributions. Table 11 shows that conflict counts vary smoothly with the two most consequential parameters. Production deployments will nonetheless encounter organizational specifics the generator does not anticipate. This is the principal external-validity gap.

Production graph database comparison. Table 8 compares T-RKG against flat-list and SQLite alternatives on the same data and reports a 121 / 406 propagation speedup. It does not compare against Neo4j, Amazon Neptune, JanusGraph, or any embedding-based KG store. The qualitative conclusion (graph-native traversal beats relational-CTE) is not expected to flip on those platforms. Absolute numbers will reshape.

Ontology coverage. Fourteen pairwise conflict rules across nine regulations represent a subset of the US-global compliance space. FERPA, CIPA, FINRA Rule 4511 in its granular detail, and emerging regulations such as the EU AI Act's data governance provisions are not included.

Explicit relationship assumption. T-RKG reasons over relationships that source systems have formally recorded. Implicit or semantic relationships between records are outside the system's current scope. NLP-based relationship extraction is an identified priority extension.

Single-jurisdiction record assignment. The generator assigns each record a single primary jurisdiction. Records subject to multiple overlapping regulatory regimes would generate additional conflict instances of the Jurisdiction type.

9. Conclusion

Enterprise records governance is a problem for which the technical infrastructure currently deployed in most organizations is inadequate in a specific, structural sense. The inadequacy is not primarily one of scale or speed. It is one of knowledge. Siloed systems enforce the regulations they know about, on the records they can see. They have no view of the obligations that other systems are simultaneously enforcing on the same underlying information objects. Conflicts between those obligations remain undetected.

T-RKG addresses this by treating governance as a knowledge representation problem. The formal ontology (47 classes across five modules, 23 object properties; supplied as a Turtle file) encodes regulation profiles, relationship semantics, jurisdictional hierarchies, and conflict type definitions as typed entities with queryable properties. Records are modeled as composite governance entities that aggregate attributes from multiple source systems, not as content objects. Hold propagation traverses this structure under semantic constraints. Conflict detection reasons over it through applicability inference. The combination yields governance capabilities (cross-domain conflict identification, semantically justified hold propagation, temporal point-in-time query answering) that no architecture lacking unified regulatory knowledge can replicate.

The experimental record. T-RKG detects 55 to 4,672 conflicts per dataset, of which 4 to 831 are cross-domain against zero cross-domain in both baselines. The conservative propagation policy yields a 3.36 mean hold-coverage expansion. The ablation confirms that ontological reasoning, typed relationship filtering, and graph propagation each contribute independently. Per-record applicability F1 is 0.581 (Siloed) and 0.113 (No-Ontology) against an ontologically constructed reference, and T-RKG F1 of 0.823 plus or minus 0.011 under a noised-input regime where the system must recover from 10 percent per-attribute label noise. Conflict detection time is 321 plus or minus 2 ms at 100K records. Sub-millisecond propagation latency holds at matter-scoped seed sets and scales to 13.7 ms at 5,000 seeds. Propagation is 121 / 406 times faster than flat-list / SQLite alternatives on identical data. A parameter sweep shows smooth, monotonic behavior of the cross-domain conflict count as the EU jurisdictional fraction varies from 5 to 40 percent.

Substantial work remains. Production validation against real record populations is the most pressing gap. The ontology requires extension to additional regulatory frameworks. A direct performance comparison against production graph databases is identified as the next logical extension. Relationship extraction from source systems with incomplete or inconsistently structured metadata is an engineering challenge whose scope cannot be fully characterized without deployment experience. What the present work establishes is that the foundational knowledge representation approach is sound, the capabilities it enables are not available by other means, and the system degrades gracefully under both data-quality stress and parameter variation.

References

- [1] The Sedona Conference, "Commentary on Legal Holds, Second Edition: The Trigger and the Process," *The Sedona Conference Journal*, vol. 20, pp. 341–398, 2019.
- [2] EDRM, "Electronic Discovery Reference Model," 2020. <https://edrm.net>
- [3] ISO, "ISO 15489-1:2016 Information and Documentation: Records Management," ISO, 2016.
- [4] M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, H. T. Cory, "The British Nationality Act as a Logic Program," *Communications of the ACM*, vol. 29, no. 5, pp. 370–386, 1986.
- [5] K. D. Ashley, *Artificial Intelligence and Legal Analytics*. Cambridge University Press, 2017.
- [6] M. Palmirani, G. Governatori, A. Rotolo, S. Tabet, H. Boley, A. Paschke, "LegalRuleML: XML-Based Rules and Norms," in *RuleML, LNCS 7018*, Springer, 2011, pp. 298–312.
- [7] R. Hoekstra, J. Breuker, M. Di Bello, A. Boer, "The LKIF Core Ontology of Basic Legal Concepts," in *LOAIT*, 2007, pp. 43–63.
- [8] M. Weber et al., "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," in *KDD Workshop on Anomaly Detection in Finance*, 2019.
- [9] M. Bennett, "The Financial Industry Business Ontology," *Semantic Technology Conference*, 2013.
- [10] D. W. Arner, J. Barberis, R. P. Buckley, "FinTech, RegTech, and the Reconceptualization of Financial Regulation," *Northwestern Journal of International Law and Business*, vol. 37, no. 3, pp. 371–413, 2017.
- [11] I. Chalkidis et al., "LEGAL-BERT: The Muppets Straight out of Law School," in *Findings of EMNLP*, 2020, pp. 2898–2904.
- [12] M. Galkin, S. Auer, M.-E. Vidal, S. Scerri, "Enterprise Knowledge Graphs: A Semantic Approach for Knowledge Management in the Next Generation of Enterprise Information Systems," in *Proc. ICEIS*, vol. 2, 2017, pp. 88–98.
- [13] N. Noy, Y. Gao, A. Jain, A. Narang, A. Patterson, J. Taylor, "Industry-Scale Knowledge Graphs," *Communications of the ACM*, vol. 62, no. 8, pp. 36–43, 2019.
- [14] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE TKDE*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [15] A. Meroño-Peñuela, E. Simperl, A. Kurteva, I. Reklos, "KG.GOV: Knowledge Graphs as the Backbone of Data Governance in AI," *Journal of Web Semantics*, vol. 85, p. 100847, 2025.
- [16] C. Gutierrez, C. A. Hurtado, A. Vaisman, "Introducing Time into RDF," *IEEE TKDE*, vol. 19, no. 2, pp. 207–218, 2007.
- [17] J. F. Allen, "Maintaining Knowledge about Temporal Intervals," *Communications of the ACM*, vol. 26, no. 11, pp. 832–843, 1983.
- [18] J. Zhu, Y. Fu, J. Zhou, D. Chen, "MGHGN: Multi-Granularity History Graph Network for Temporal Knowledge Graph Reasoning," *Data and Knowledge Engineering*, vol. 160, p. 102496, 2025.
- [19] L. Bai, M. Chen, L. Zhu, X. Meng, "Multi-Hop Temporal Knowledge Graph Reasoning with Temporal Path Rules Guidance," *Expert Systems with Applications*, vol. 223, p. 119804, 2023.

- [20] H. Cai, Y. Liao, R. Li, "Temporal Knowledge Graph Completion: A Survey," arXiv:2308.02457, 2023.
- [21] T. R. Gruber, "Toward Principles for the Design of Ontologies Used for Knowledge Sharing," *International Journal of Human-Computer Studies*, vol. 43, no. 5–6, pp. 907–928, 1995.
- [22] N. Guarino, D. Oberle, S. Staab, "What is an Ontology?" in *Handbook on Ontologies*, 2nd ed. Springer, 2009.
- [23] M. Uschold, M. King, S. Moralee, Y. Zorgios, "The Enterprise Ontology," *Knowledge Engineering Review*, vol. 13, no. 1, pp. 31–89, 1998.
- [24] M. Grüninger, M. S. Fox, "The Role of Competency Questions in Enterprise Ontology Design," in *Benchmarking*, Springer, 1995.
- [25] L. M. Schriml et al., "Human Disease Ontology 2018 Update: Classification, Content and Workflow Expansion," *Nucleic Acids Research*, vol. 47, no. D1, pp. D955–D962, 2019.
- [26] M. Ringsquandl, S. Lamparter, S. Brandt, T. Hubauer, R. Lepratti, "Semantic-Guided Feature Selection for Industrial Automation Systems," in *The Semantic Web (ISWC 2015)*, LNCS 9366, Springer, 2015, pp. 225–240.
- [27] N. Hubert, P. Monnin, M. d'Aquin, D. Monticolo, A. Brun, "PyGraft: Configurable Generation of Synthetic Schemas and Knowledge Graphs at Your Fingertips," in *Proc. ESWC*, 2024.
- [28] W. M. P. van der Aalst, *Process Mining: Data Science in Action*, 2nd ed. Springer, 2016.
- [29] Veritas Technologies, "Global Databerg Report," 2016.
<https://www.veritas.com/news-releases/2016-03-15-veritas-global-databerg-report-finds-85-percent-of-stored-data>
- [30] D. Arnott, G. Pervan, "A Critical Analysis of Decision Support Systems Research," *Journal of Information Technology*, vol. 20, no. 2, pp. 67–87, 2005.
- [31] The Sedona Conference, "Commentary on Proportionality in Electronic Discovery," *The Sedona Conference Journal*, vol. 18, pp. 141–188, 2017.
- [32] A. A. Hagberg, D. A. Schult, P. J. Swart, "Exploring Network Structure with NetworkX," in *SciPy*, 2008.
- [33] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, J. E. L. Gayo, S. Kirrane, S. Neumaier, A. Polleres, R. Navigli, A. N. Ngomo, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, A. Zimmermann, "Knowledge Graphs," *ACM Computing Surveys*, vol. 54, no. 4, Art. 71, 2021.
- [34] C. Bizer, T. Heath, T. Berners-Lee, "Linked Data: The Story So Far," *International Journal on Semantic Web and Information Systems*, vol. 5, no. 3, 2009.
- [35] J. Brank, M. Grobelnik, D. Mladenić, "A Survey of Ontology Evaluation Techniques," in *Proc. SiKDD*, 2005.
- [36] EDRM, "Information Governance Reference Model (IGRM) v3.0," 2014.
<https://edrm.net/resources/frameworks-and-standards/igrm-model/>
- [37] H. Knublauch, D. Kontokostas, "Shapes Constraint Language (SHACL)," *W3C Recommendation*, 2017.

- [38] B. Motik et al., "OWL 2 Web Ontology Language Profiles," W3C Recommendation, 2012.
- [39] *Zubulake v. UBS Warburg LLC*, 229 F.R.D. 422 (S.D.N.Y. 2004) (Scheindlin, J.). Foundational legal-hold case law.
- [40] E. Prud'hommeaux, A. Seaborne (eds.), "SPARQL Query Language for RDF," W3C Recommendation, 2008.
- [41] T. Berners-Lee, "Linked Data: Design Issues," W3C Personal Note, 2006/2009.
- [42] G. Antoniou, F. van Harmelen, *A Semantic Web Primer*, 3rd ed. MIT Press, 2012.

Author Contributions, Competing Interests, Data and Code Availability, and Generative AI Disclosure

Author contributions (CRediT). Author Name: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing (original draft), Writing (review and editing), Visualization, Supervision.

Declaration of competing interests. The author declares no competing financial or personal interests.

Data and code availability. All code, the OWL/Turtle ontology, the synthetic data generator, and the experiment runner that produces every number in this paper are provided in the supplementary repository: ``trkg/`, `experiments/run_all.py`, `ontology/trkg.ttl``. Reproduction is ``python -m experiments.run_all``.

Generative AI disclosure. During the preparation of this work the author used a generative AI assistant for code refactoring suggestions, drafting of selected prose passages, and assistance in formatting tables and figures. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article. No generative AI was used to design experiments, run measurements, derive proofs, or to make claims of fact; all numerical results were produced by the author's own deterministic experimental pipeline (``experiments/run_all.py``) and verified against the supplementary repository.